

GIVEAWAY · FROM THE REEL

Ponytail kit

AI se kam aur saaf code likhwao. **Ponytail** by DietrichGebert is a free, open-source (MIT) skill that makes your AI coding agent think like "the laziest senior dev in the room". Repo: github.com/DietrichGebert/ponytail

OPTION 1

Plugin install (Claude Code)

Send these as **two separate prompts** inside Claude Code:

```
/plugin marketplace add DietrichGebert/ponytail
```

```
/plugin install ponytail@ponytail
```

Codex, Copilot CLI, Cursor and more: the install steps for each tool are in the [repo README](#).

OPTION 2 · COMPANY BLOCKS OUTSIDE PLUGINS

Paste this into CLAUDE.md

Put it in `CLAUDE.md` at your project root (or `~/.claude/CLAUDE.md` for all projects).

```
## Write the minimum code (lazy senior dev rule)
```

Before writing any code, stop at the FIRST step that solves the task:

1. Does this need to exist? If not, skip it and say why.
2. Is it already in this codebase? Reuse it, don't rewrite it.
3. Does the language's standard library do it? Use that.
4. Does the platform do it natively (`<input type="date">`, `Intl.NumberFormat`, `structuredClone`)? Use that.
5. Does an already-installed dependency do it? Use that. Never add a new package for one function.
6. Can it be one line? Write one line.
7. Only then: write the minimum that works.

Read the code you are changing first. Be lazy about the solution, never about understanding the problem.

Never cut: input validation at trust boundaries, error handling that prevents data loss, security, accessibility.

CODEX · CURSOR · COPILOT

Same text in AGENTS.md

Save the same block as `AGENTS.md` at your project root. These tools read it automatically.

CLAUDE SKILL

SKILL.md

Save as `.claude/skills/lazy-senior/SKILL.md` in your project:

```
---
name: lazy-senior
description: Use before writing or changing code. Makes you pick the smallest
correct solution (reuse, stdlib, native feature, installed dependency) before
writing new code, without dropping validation, error handling, security or
accessibility.
---

## Write the minimum code (lazy senior dev rule)

Before writing any code, stop at the FIRST step that solves the task:

1. Does this need to exist? If not, skip it and say why.
2. Is it already in this codebase? Reuse it, don't rewrite it.
3. Does the language's standard library do it? Use that.
4. Does the platform do it natively (<input type="date">, Intl.NumberFormat,
structuredClone)? Use that.
5. Does an already-installed dependency do it? Use that. Never add a new package
for one function.
6. Can it be one line? Write one line.
7. Only then: write the minimum that works.

Read the code you are changing first. Be lazy about the solution, never about
understanding the problem.

Never cut: input validation at trust boundaries, error handling that prevents data
loss, security, accessibility.
```

HONEST NOTES

Read before you trust the numbers

- 54% less code, 20% cheaper, 27% faster and "safety tests passed" are the **creator's own benchmark**: 12 feature tasks on one FastAPI + React repo, Claude Haiku 4.5, n=4. Not a guarantee for your project.
- Savings are big where AI over-builds (colour picker 287 → 23 lines) and near zero where the code is already minimal.
- The pasted rule is a lighter version of the plugin. The plugin also runs hooks that keep it active every session.
- The README says on some models cost can go up.

SOURCES

Links

- [Ponytail repo](#)
- [Creator's benchmark write-up](#)
- [The public critique \(Scott Logic\)](#)
- [InfoQ on the fix](#)

Rule text adapted from Ponytail's decision ladder (MIT licence, © DietrichGebert). Made by @shashikant.ai